

Student Locker Assignment System

Course: Database Management Systems

Instructor: Dr. Ashraf Odeh

Student Name: [Mohamed Abdulrahman Younis]

Student ID: [202211264]

Submission Date: [1/12/2025]

Table of Contents

Student Locker Assignment System	1
1. Scenario	2
2. ER Diagram	3
3. EER Diagram	3
4. Mapping	4
5. Relation Constraints	4
6. Relational Algebra	6
7. Apply Functional Dependencies	7
8. Apply Database Normalization	7
9. Implement SQL Statements	8

1. Scenario

The "Student Locker Assignment System" is designed to efficiently manage the allocation of lockers to students in universities. Each locker is uniquely identified, and every locker must be assigned to a student. Students are categorized into Undergraduate Students and Graduate Students, while lockers are generalized into Storage Units to accommodate different types of storage. However, not all students may require a locker. The system ensures that all locker assignments are recorded with the date of assignment, maintaining clear records for administrative purposes.

To ensure the system is complete, an **Admin** entity has been added to manage the registration of students, lockers, and assignments. The **Admin** entity is responsible for recording and maintaining the data integrity of the system.

Entities and Attributes:

1. Student:

- **Attributes:** StudentID (PK), Name, GPA.

2. UndergraduateStudent (Specialization of Student):

- **Attributes:** StudentID (PK), Major, Year.

3. GraduateStudent (Specialization of Student):

- **Attributes:** StudentID (PK), ThesisTopic, Advisor.

4. StorageUnit (Generalization of Locker):

- **Attributes:** UnitID (PK), Location, Size.

5. Locker:

- **Attributes:** LockerID (PK), UnitID (FK), LockerType (Shared, Private).

6. Assignment (Weak Entity):

- **Attributes:** StudentID (FK), LockerID (FK), AssignmentDate.

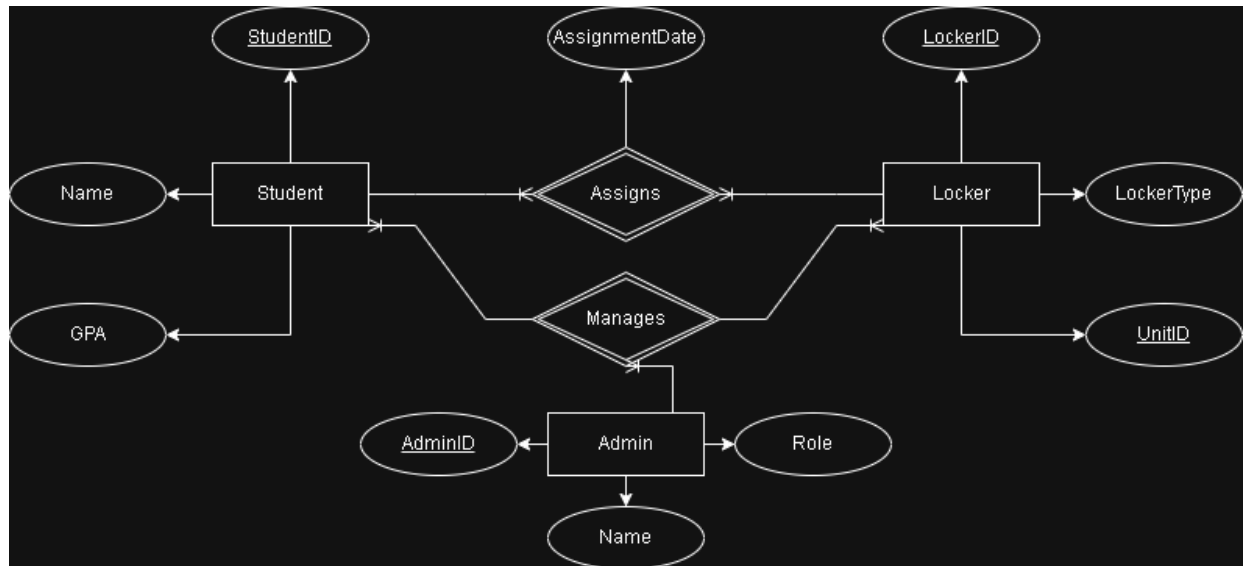
7. Admin:

- **Attributes:** AdminID (PK), Name, Role.

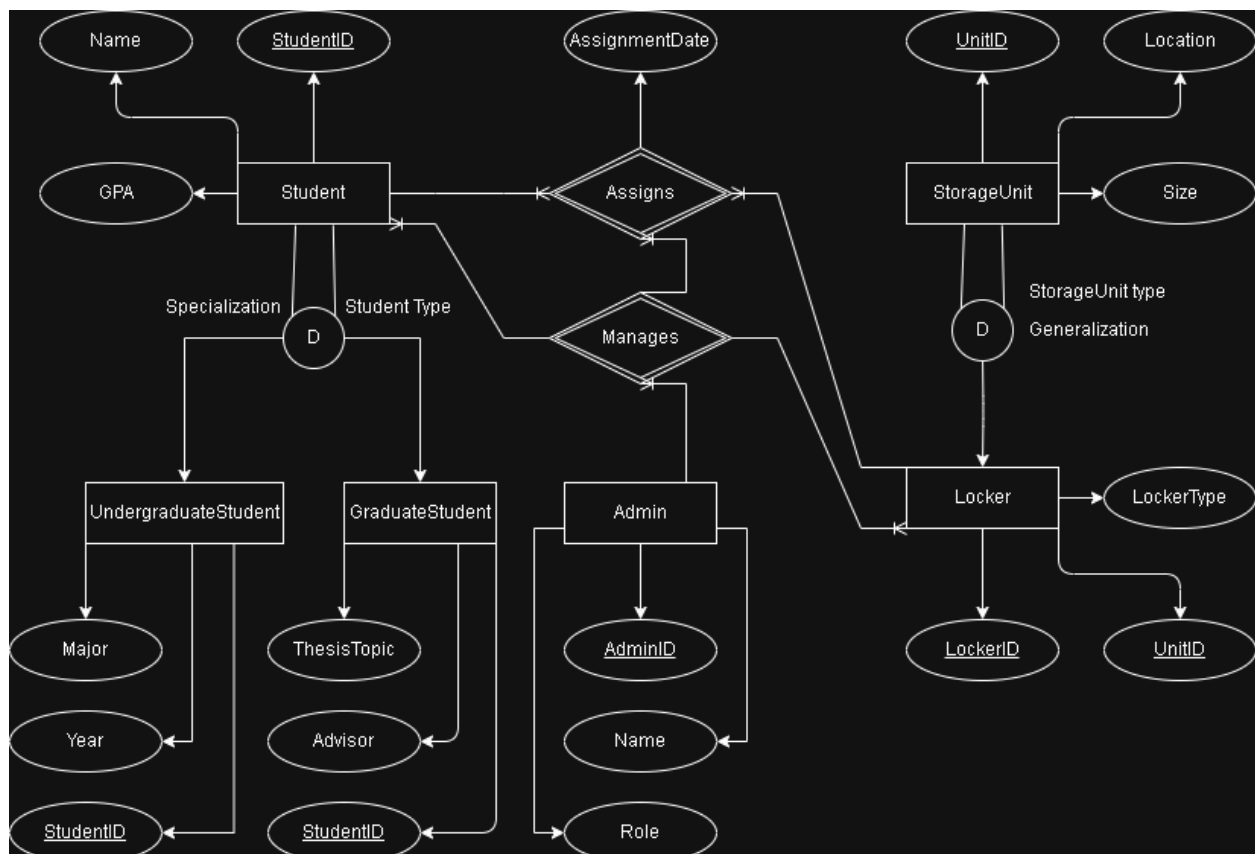
Relationships:

- **Student assigns to Locker through Assignment.**
 - **Cardinality:** 0:N (Student may not have a locker, but not vice versa).
- **Locker belongs to StorageUnit.**
- **Student specializes into UndergraduateStudent and GraduateStudent.**
- **Admin manages Student, Locker, and Assignment.**

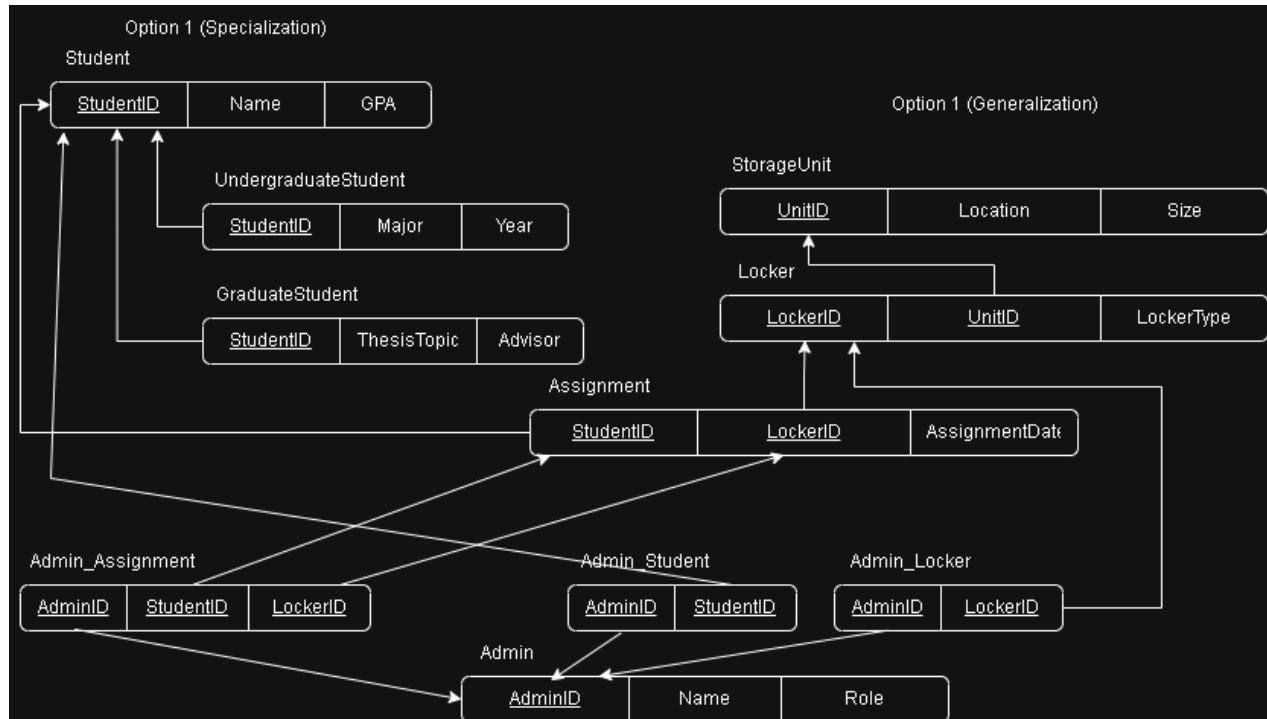
2. ER Diagram



3. EER Diagram



4. Mapping



5. Relation Constraints

Key Constraints

1. Primary Key:

- Each entity has a unique identifier:
 - StudentID for Student, UndergraduateStudent, and GraduateStudent.
 - UnitID for StorageUnit.
 - LockerID for Locker.
 - AdminID for Admin.
- A **Primary Key** cannot contain null values.

2. Composite Key:

- The Assignment table uses a composite key (StudentID, LockerID) to ensure unique locker assignments.

3. Foreign Key:

- StudentID in Assignment references StudentID in Student.
- LockerID in Assignment references LockerID in Locker.

- UnitID in Locker references UnitID in StorageUnit.
- AdminID in Admin references the entities it manages.

Entity Integrity Constraints

- The Primary Key of each entity must not contain null values, ensuring that every tuple is uniquely identifiable.

Referential Integrity Constraints

- **Foreign Key Constraints** ensure valid relationships between entities:
 - StudentID in Assignment must match a valid StudentID in Student.
 - LockerID in Assignment must match a valid LockerID in Locker.
 - UnitID in Locker must match a valid UnitID in StorageUnit.
 - AdminID must references valid entities it manages.

Domain Constraints

- Each attribute must have values within a specific domain. For example:
 - GPA in Student must be a number between 0 and 100.
 - LockerType in Locker must be either 'Shared' or 'Private'.
 - Size in StorageUnit must be a valid size category (e.g., Small, Medium, Large).

Participation Constraints

1. Total Participation:

- Every locker must belong to a storage unit. All LockerID values in Locker must have a corresponding UnitID in StorageUnit.

2. Partial Participation:

- Not all students require a locker. Some StudentID values in Student may not have a corresponding entry in Assignment.

Generalization and Specialization

1. Specialization:

- The Student entity is specialized into two sub-entities:

- UndergraduateStudent with attributes: StudentID, Major, Year.
- GraduateStudent with attributes: StudentID, ThesisTopic, Advisor.
- **Disjoint Constraint:** A student can either be an UndergraduateStudent or a GraduateStudent, but not both.
- **Participation Constraint:** Specialization is total, meaning all students must belong to a specialized subclass.

2. Generalization:

- The Locker entity is generalized into a higher-level entity StorageUnit with attributes: UnitID, Location, Size.
- Locker inherits from StorageUnit and adds the attribute LockerType ('Shared' or 'Private').

Semantic Integrity Constraints

- A locker can only be assigned to one student at a time.

Actions for Constraint Violations

1. Insert, Update, and Delete Operations:

- Violations of constraints can occur during these operations. The following actions can be taken:
 - **Reject:** The operation is rejected, and the user is informed.
 - **Cascade:** The operation is propagated to related tuples.
 - **Set Null:** The affected attributes are set to null.

6. Relational Algebra

1. Retrieve all students with a GPA above 3.0 assigned to lockers

$\pi(\text{StudentID}, \text{Name}, \text{GPA}) (\sigma(\text{GPA} > 3.0) (\text{Student}) \bowtie \text{Assignment} \bowtie \text{Locker})$

2. Find all lockers in 'Building A' managed by a specific admin:

$\pi(\text{LockerID}, \text{LockerType}) (\sigma(\text{Location} = \text{'Building A'}) (\text{StorageUnit}) \bowtie \text{Locker} \bowtie \text{Admin_Locker} \bowtie \sigma(\text{AdminID} = 1) (\text{Admin}))$

3. Rename *GraduateStudent* and list graduate students with their thesis topics

$\pi(\text{StudentID}, \text{Name}, \text{ThesisTopic}) (\text{Student} \bowtie \rho(\text{GS_Thesis}(\text{ThesisTopic}, \text{Advisor}, \text{StudentID})) (\text{GraduateStudent}))$

4. **Retrieve all storage units and lockers, excluding 'Small' storage units:**

$\pi(\text{UnitID}, \text{Location}, \text{Size}, \text{LockerID}, \text{LockerType}) ((\sigma(\text{Size} \neq \text{'Small'}) (\text{StorageUnit})) \bowtie \text{Locker})$

7. Apply Functional Dependencies

1. **Student Table:**

- $\text{StudentID} \rightarrow \text{Name}, \text{GPA}$
- $\text{StudentID} \rightarrow \text{Major}, \text{Year}$ (for UndergraduateStudent)
- $\text{StudentID} \rightarrow \text{ThesisTopic}, \text{Advisor}$ (for GraduateStudent)

2. **Locker Table:**

- $\text{LockerID} \rightarrow \text{UnitID}, \text{LockerType}$

3. **StorageUnit Table:**

- $\text{UnitID} \rightarrow \text{Location}, \text{Size}$

4. **Assignment Table:**

- $\text{StudentID}, \text{LockerID} \rightarrow \text{AssignmentDate}$

5. **Admin Table:**

- $\text{AdminID} \rightarrow \text{Name}, \text{Role}$

8. Apply Database Normalization

The database is already normalized to **3NF**. Here's the breakdown:

1. **1NF:**

- All attributes are atomic (e.g., Name, GPA, LockerType).

2. **2NF:**

- All non-prime attributes are fully functionally dependent on the primary key.
- Example: In Assignment, AssignmentDate depends on the composite key (StudentID, LockerID).

3. **3NF:**

- No transitive dependencies exist.
- Example: In Locker, LockerType depends only on LockerID, not on UnitID.

9. Implement SQL Statements

1. Creating Tables

```
CREATE TABLE Student (  
    StudentID NUMBER PRIMARY KEY,  
    Name VARCHAR2(100) NOT NULL,  
    GPA NUMBER(3, 2) CHECK (GPA BETWEEN 0.00 AND 4.00)  
);
```

```
CREATE TABLE UndergraduateStudent (  
    StudentID NUMBER PRIMARY KEY,  
    Major VARCHAR2(50) NOT NULL,  
    Year NUMBER CHECK (Year BETWEEN 1 AND 4),  
    CONSTRAINT fk_ug_student FOREIGN KEY (StudentID) REFERENCES  
    Student(StudentID)  
);
```

```
CREATE TABLE GraduateStudent (  
    StudentID NUMBER PRIMARY KEY,  
    ThesisTopic VARCHAR2(255),  
    Advisor VARCHAR2(100),  
    CONSTRAINT fk_grad_student FOREIGN KEY (StudentID) REFERENCES  
    Student(StudentID)  
);
```

```
CREATE TABLE StorageUnit (  
    UnitID NUMBER PRIMARY KEY,  
    Location VARCHAR2(100) NOT NULL,  
    Size VARCHAR2(20) CHECK (Size IN ('Small', 'Medium', 'Large'))  
);
```

```
CREATE TABLE Locker (  
    LockerID NUMBER PRIMARY KEY,  
    UnitID NUMBER NOT NULL,  
    LockerType VARCHAR2(10) CHECK (LockerType IN ('Shared', 'Private')),  
    CONSTRAINT fk_locker_unit FOREIGN KEY (UnitID) REFERENCES  
    StorageUnit(UnitID)  
);
```

```
CREATE TABLE Assignment (  
    StudentID NUMBER NOT NULL,  
    LockerID NUMBER NOT NULL,  
    AssignmentDate DATE NOT NULL,  
    CONSTRAINT pk_assignment PRIMARY KEY (StudentID, LockerID),  
    CONSTRAINT fk_assignment_student FOREIGN KEY (StudentID) REFERENCES  
    Student(StudentID),
```



```

        CONSTRAINT fk_assignment_locker FOREIGN KEY (LockerID) REFERENCES
Locker(LockerID)
);

CREATE TABLE Admin (
    AdminID NUMBER PRIMARY KEY,
    Name VARCHAR2(100) NOT NULL,
    Role VARCHAR2(50) NOT NULL
);

CREATE TABLE Admin_Student (
    AdminID NUMBER NOT NULL,
    StudentID NUMBER NOT NULL,
    CONSTRAINT pk_admin_student PRIMARY KEY (AdminID, StudentID),
    CONSTRAINT fk_admin_student_admin FOREIGN KEY (AdminID) REFERENCES
Admin(AdminID),
    CONSTRAINT fk_admin_student_student FOREIGN KEY (StudentID) REFERENCES
Student(StudentID)
);

CREATE TABLE Admin_Locker (
    AdminID NUMBER NOT NULL,
    LockerID NUMBER NOT NULL,
    CONSTRAINT pk_admin_locker PRIMARY KEY (AdminID, LockerID),
    CONSTRAINT fk_admin_locker_admin FOREIGN KEY (AdminID) REFERENCES
Admin(AdminID),
    CONSTRAINT fk_admin_locker_locker FOREIGN KEY (LockerID) REFERENCES
Locker(LockerID)
);

CREATE TABLE Admin_Assignment (
    AdminID NUMBER NOT NULL,
    StudentID NUMBER NOT NULL,
    LockerID NUMBER NOT NULL,
    CONSTRAINT pk_admin_assignment PRIMARY KEY (AdminID, StudentID,
LockerID),
    CONSTRAINT fk_admin_assignment_admin FOREIGN KEY (AdminID)
REFERENCES Admin(AdminID),
    CONSTRAINT fk_admin_assignment_student FOREIGN KEY (StudentID)
REFERENCES Student(StudentID),
    CONSTRAINT fk_admin_assignment_locker FOREIGN KEY (LockerID)
REFERENCES Locker(LockerID)
);

```

2. Data Insertion

```
INSERT INTO Student (StudentID, Name, GPA) VALUES (1, 'John Doe', 3.85);
INSERT INTO UndergraduateStudent (StudentID, Major, Year) VALUES (1, 'Computer Science', 3);
```

```
INSERT INTO StorageUnit (UnitID, Location, Size) VALUES (101, 'Building A', 'Medium');
INSERT INTO Locker (LockerID, UnitID, LockerType) VALUES (201, 101, 'Private');
```

```
INSERT INTO Assignment (StudentID, LockerID, AssignmentDate) VALUES (1, 201, '2024-12-01');
```

```
INSERT INTO Admin (AdminID, Name, Role) VALUES (1, 'Admin User', 'Manager');
INSERT INTO Admin_Student (AdminID, StudentID) VALUES (1, 1);
INSERT INTO Admin_Locker (AdminID, LockerID) VALUES (1, 201);
INSERT INTO Admin_Assignment (AdminID, StudentID, LockerID) VALUES (1, 1, 201);
```

3. Query Data

Retrieve all students assigned to lockers:

```
SELECT S.StudentID, S.Name, S.GPA, A.AssignmentDate
FROM Student S
JOIN Assignment A ON S.StudentID = A.StudentID;
```

Find all lockers managed by a specific admin:

```
SELECT L.LockerID, L.LockerType
FROM Admin_Locker AL
JOIN Locker L ON AL.LockerID = L.LockerID
WHERE AL.AdminID = 1;
```

List all graduate students with their thesis topics:

```
SELECT S.StudentID, S.Name, G.ThesisTopic
FROM Student S
JOIN GraduateStudent G ON S.StudentID = G.StudentID;
```

Retrieve all storage units with their lockers:

```
SELECT SU.UnitID, SU.Location, SU.Size, L.LockerID, L.LockerType
FROM StorageUnit SU
JOIN Locker L ON SU.UnitID = L.UnitID;
```